

Laboratorio 2 - Redes Neuronales Convolucionales

Diego Fabian Morales Davila

Repositorio: <https://github.com/FabianMoraeles/DeepLab2>

1. Investigación de las capas de PyTorch para CNN

nn.Conv2d

Aplica una convolución 2D, deslizando un conjunto de filtros llamados kernels sobre la imagen para producir mapas de características. Parametros clave: `in_channels`, `out_channels`, `kernel_size`, `stride` y `padding`. Cada filtro comparte los mismos pesos en toda la imagen, detectando el mismo patrón sin importar donde aparezca.

nn.MaxPool2d

Reduce la dimensión espacial de un mapa de características quedándose con el valor máximo de cada ventana, definida por `kernel_size` y `stride`. Baja el costo computacional, aporta invarianza a pequeñas traslaciones y ayuda a evitar sobreajuste.

nn.AvgPool2d

Igual que `MaxPool2d` pero usando el promedio de la ventana en vez del máximo. Suaviza la representación en vez de resaltar la activación más fuerte.

nn.BatchNorm2d

Normaliza las activaciones de cada canal a lo largo del batch y aplica una escala y un desplazamiento aprendibles, definidos por `num_features`. Estabiliza y acelera el entrenamiento y actúa además como un regularizador leve.

nn.Flatten

Convierte el mapa de características de salida de las capas convolucionales, con forma canales por alto por ancho, en un vector unidimensional. Es el puente entre las capas convolucionales, que trabajan con tensores 2D, y las capas totalmente conectadas.

nn.CrossEntropyLoss

Perdida para clasificación multiclase. Combina un softmax sobre los logits de salida con la perdida de log-verosimilitud negativa, por lo que la red no debe aplicar softmax en su última capa. Recibe logits sin normalizar y el índice entero de la clase correcta.

Tensores en PyTorch

Un tensor es la estructura de datos básica de PyTorch, un arreglo multidimensional similar a un `ndarray` de `numpy` pero con soporte nativo para GPU y para el cálculo automático de gradientes, `autograd`.

Campo receptivo o receptive field

Es la región de la imagen de entrada que influye en el valor de una unidad de un mapa de características. Al apilar más capas convolucionales o de pooling, el campo receptivo efectivo crece, permitiendo que unidades más profundas respondan a patrones que abarcan una porción mayor de la imagen.

Por qué una CNN requiere menos parámetros que un MLP equivalente

Un MLP necesita un peso independiente por cada combinación de pixel de entrada y neurona oculta. En una CNN, cada filtro comparte el mismo conjunto pequeño de pesos en todas las posiciones de la imagen, por lo que el número de parámetros depende del tamaño del kernel y del número de canales, no del tamaño de la imagen. Esto permite capturar patrones espaciales locales con muchos menos parámetros que un MLP equivalente.

2. Resultados de las iteraciones

MLP, 8 iteraciones, métricas sobre validación

#	Arquitectura	Activo	Optim./LR	Batch/Ep.	Regularización	Params	Acc_val	F1_val
1	[128,64]	ReLU	Adam/0.001	128/8	ninguna	109386	0.9719	0.9719
2	[256,128,64]	ReLU	Adam/0.001	128/8	ninguna	242762	0.9698	0.9697
3	[64]	ReLU	Adam/0.001	128/8	ninguna	50890	0.9643	0.9641
4	[128,64]	LeakyReLU	Adam/0.001	128/8	ninguna	109386	0.9708	0.9707
5	[128,64]	ReLU	SGD momentum/0.01	128/8	ninguna	109386	0.9625	0.9624
6	[128,64]	ReLU	Adam/0.001	32/8	ninguna	109386	0.9769	0.9768
7	[128,64]	ReLU	Adam/0.001	128/8	dropout=0.3	109386	0.9728	0.9726
8	[128,64]	ReLU	Adam/0.001	128/8	L2=1e-4, BatchNorm1d	109770	0.9772	0.9771

CNN, 8 iteraciones, métricas sobre validación

	Filtros	Pool	Optim./LR	Batch/Ep.	Regularizacion	Params	Acc_val	F1_val
9	[16,32]	max	Adam/0.001	128/5	ninguna	206922	0.9848	0.9845
10	[32,64]	max	Adam/0.001	128/5	ninguna	421642	0.9882	0.9881
11	[16,32,64]	max	Adam/0.001	128/5	ninguna	98442	0.9856	0.9855
12	[16,32]	avg	Adam/0.001	128/5	ninguna	206922	0.9805	0.9803
13	[16,32]	max	Adam/0.001	128/5	BatchNorm2d	207018	0.9853	0.9852
14	[16,32]	max	SGD momentum/0.01	128/5	ninguna	206922	0.9795	0.9793
15	[16,32]	max	Adam/0.001	32/5	ninguna	206922	0.9876	0.9874
16	[16,32]	max	Adam/0.001	128/5	dropout=0.4	206922	0.986	0.9859

3. Comparación de arquitecturas

Se selecciono la mejor configuración de cada arquitectura según el F1-score macro de validación, MLP iteración 8 y CNN iteración 10, y se evaluó cada una una única vez sobre el conjunto de prueba:

Arquitectura	Iteracion	Parametros	Tiempo entren. s	Acc_test	Prec_test	Rec_test	F1_test
MLP	8	109770	6.6	0.9788	0.9789	0.9786	0.9787
CNN	10	421642	33	0.9897	0.9898	0.9896	0.9896

La CNN obtiene mejor desempeño en las cuatro métricas de prueba, con accuracy de 0.9897 frente a 0.9788 del MLP. La mejor CNN tiene casi 4 veces más parámetros que el mejor MLP, 421642 frente a 109770, pero varias CNN con menos parámetros ya superan a todas las iteraciones del MLP, incluyendo a la más grande de todas, iteración 2, con 242762 parámetros y accuracy de validación de solo 0.9698. Esto confirma que la ventaja de la CNN no depende solo del número de parámetros sino de que sus filtros compartidos explotan la estructura espacial 2D de la imagen. Como contraparte, la CNN tomo aproximadamente 5 veces más tiempo de entrenamiento que el MLP en su mejor configuración, 33.0 segundos frente a 6.6 segundos.

4. Discusión y análisis

Impacto de los cambios de hiperparametros

En el MLP, el mayor impacto positivo fue combinar weight decay, L2 igual a $1e-4$, con BatchNorm1d, iteración 8, que subió el accuracy de 0.9719 a 0.9772, el mejor resultado del MLP. El mayor impacto negativo fue SGD con momentum y lr igual a 0.01, iteración 5, que bajo el accuracy a 0.9625. En la CNN, el mayor impacto positivo fue aumentar los filtros de 16 y 32 a 32 y 64, iteración 10, con 0.9882, y el mayor impacto negativo fue también SGD con momentum, iteración 14, con 0.9795.

Overfitting y underfitting

Se observo sobreajuste leve en el MLP baseline y, mas marcado, en la arquitectura más profunda, iteración 2, donde la perdida de entrenamiento seguía bajando mientras la de validación se estancaba. Underfitting se observó en la arquitectura de una sola capa, iteración 3, con ambas perdidas altas y cercanas entre sí. Dropout y weight decay con BatchNorm1d redujeron la brecha entre las curvas. La CNN mostro en general menos sobreajuste que el MLP, incluso en su configuración baseline.

Efecto de la regularización

En el MLP la regularización si mejoro el desempeño: L2 con BatchNorm1d, iteración 8, dio el mejor resultado, superando al dropout solo, iteración 7. En la CNN ninguna regularización explicita, BatchNorm2d o dropout, supero a simplemente aumentar la capacidad del modelo, iteración 10; con solo 5 epochs la CNN baseline no llego a sobre ajustar lo suficiente para que la regularización mejorara el accuracy final.

MLP vs CNN en test

La CNN obtuvo mejor desempeño en test en las cuatro metricas, accuracy 0.9897 frente a 0.9788. La CNN procesa la imagen como tensor 2D y sus filtros comparten pesos, explotando la cercanía espacial entre pixeles, mientras que el MLP aplana la imagen y trata cada pixel de forma independiente, perdiendo la relación de vecindad.

Errores más comunes

En el MLP las confusiones más frecuentes son 4 clasificado como 9, 11 casos, y 2 clasificado como 7, 10 casos. En la CNN el error más frecuente también es 9 confundido con 4, 10 casos, pero el resto de las confusiones bajan notablemente; la confusión entre 2 y 7 prácticamente desaparece. Esto es consistente con que 4 y 9 comparten trazos similares que incluso una CNN puede confundir, mientras que confusiones como 2 con 7, más relacionadas con la posición espacial de los trazos, se corrigen casi por completo con las capas convolucionales.

Modelo de producción

Se elegiría la CNN baseline, iteración 9, con 206922 parámetros y accuracy de 0.9848, muy cerca del mejor resultado de la búsqueda, 0.9882 en la iteración 10, pero con la mitad de los parámetros y un tiempo de entrenamiento menor, 15.1 segundos frente a 33.0 segundos.

5. Conclusiones

- La CNN supero al MLP en todas las métricas de prueba, accuracy, precisión, recall y F1-score, confirmando que explotar la estructura espacial de la imagen mediante convoluciones es más efectivo que tratar cada pixel de forma independiente.
- El número de parámetros por sí solo no explica el mejor desempeño: varias CNN con menos parámetros que el MLP más grande superaron a todas las iteraciones del MLP.
- La regularización ayudo más al MLP, que mostro mayor tendencia al sobreajuste, a la CNN, cuyo sesgo espacial ya limita el sobreajuste con pocas epochs.
- Para producción, priorizando exactitud y eficiencia, la CNN baseline, iteración 9, ofrece el mejor balance entre accuracy, parámetros y tiempo de entrenamiento.